Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank

Jiarui Jin[1], Xianyu Chen[1], Weinan Zhang[1], Mengyue Yang[2], Yang Wang[3], Yali Du[4], Yong Yu[1], Jun Wang[2]

[1]Shanghai Jiao Tong University, [2]University College London, [3]East China Normal University, [4]King's College London

SHANGHAI JIAO TONG UNIVERSITY

EAST CHINA NORMAL UNIVERSITY

Slides: https://jinjiarui.github.io/files/cikm23_oral.pdf

Content

- Problem Background
  - Set-to-Arrangement Framework
- Architecture
  - STARank
  - Supervisions
- Experiment
- Conclusion

Content

- **Problem Background**
  - Set-to-Arrangement Framework
- Architecture
  - STARank
  - Supervisions
- Experiment
- Conclusion

Set-to-Arrangement v.s. Learning-to-Rank

- Permutation Invariance: The output of the model must be the same under every possible permutation of the elements from the input set.
- Varying Input Length: The same model must be able to process input sets of different lengths.

  Reference: Set-to-Sequence Methods in Machine Learning: a Review. Journal of Artificial Intelligence Research 2021.

- Candidate items forming a set should require permutation invariance. Previous Papers are focusing on this:
  - SetRank uses multi-head attention networks as the encoders.

    Reference: SetRank: Learning a Permutation-Invariant Ranking Model for Information Retrieval. SIGIR 2020.

- Sequence-to-Sequence for learning-to-rank:
  - Seq2Slate uses the seq2slate pointer network architecture for ranking.

    Reference: Seq2Slate: Reranking and Slate Optimization with RNNs. arXiv 2018.

Set-to-Arrangement Framework

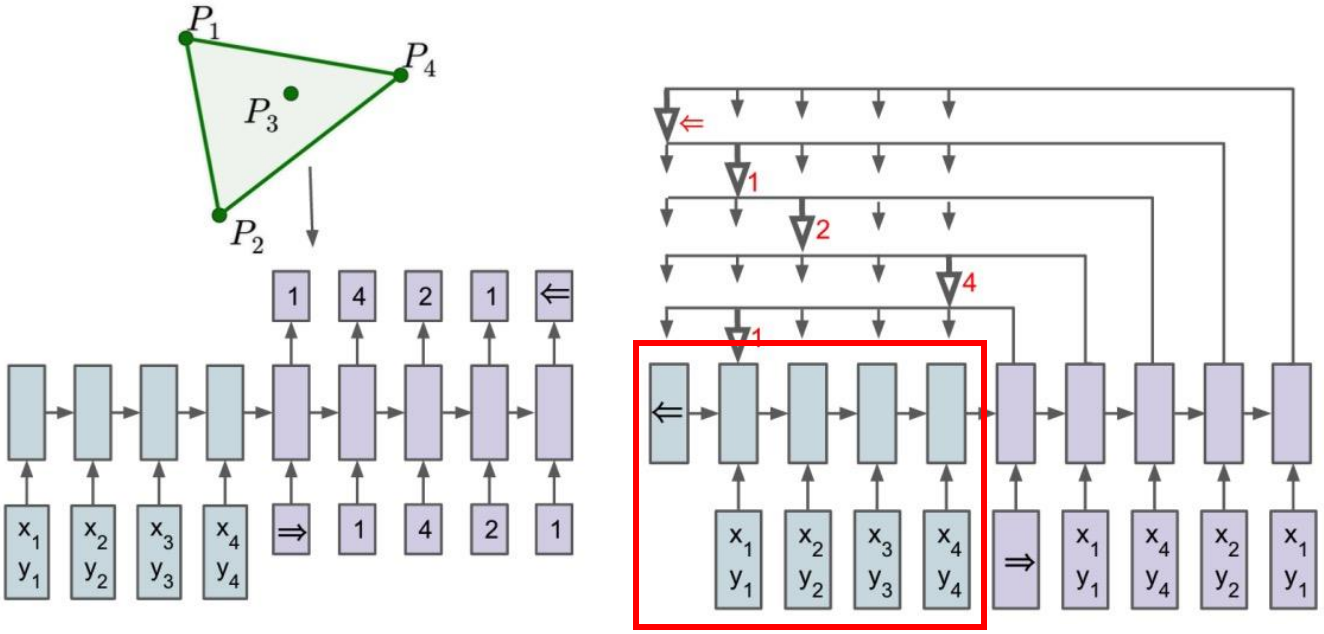

Figure: Left: sequence-to-sequence: an RNN processes the input sequence to generate the output sequence using the probability chain rule. Right: set-to-sequence: an RNN generates output sequence that modules a content-based attention mechanism over inputs. The output of the attention mechanism is a softmax distribution with dictionary size equal to the length of the input.

Reference: Pointer Networks. NeurIPS 2015.

Our Idea

- Set-to-Arrangement Framework allows the ranking models free from the sort operation: making the whole pipeline end-to-end differentiable.
- Supervisions in this regard are not user feedbacks (e.g., clicks), instead are oracle permutations of candidate items: requiring the user simulations (i.e., click models) to manage.
- User simulations also can be used as evaluation metrics, where NDCGs can be regarded as a special case of position-based click models.
- Main components: (a) Reader module: Permutation-invariant module, (b) Arranger module: Set-to-arrangement module (i.e., Plackett-Luce module), (c) Supervision module: Click models.

Content

- Problem Background
  - Set-to-Arrangement Framework
- **Architecture**
  - STARank
  - Supervisions
- Experiment
- Conclusion

Set-to-Arrangement Framework

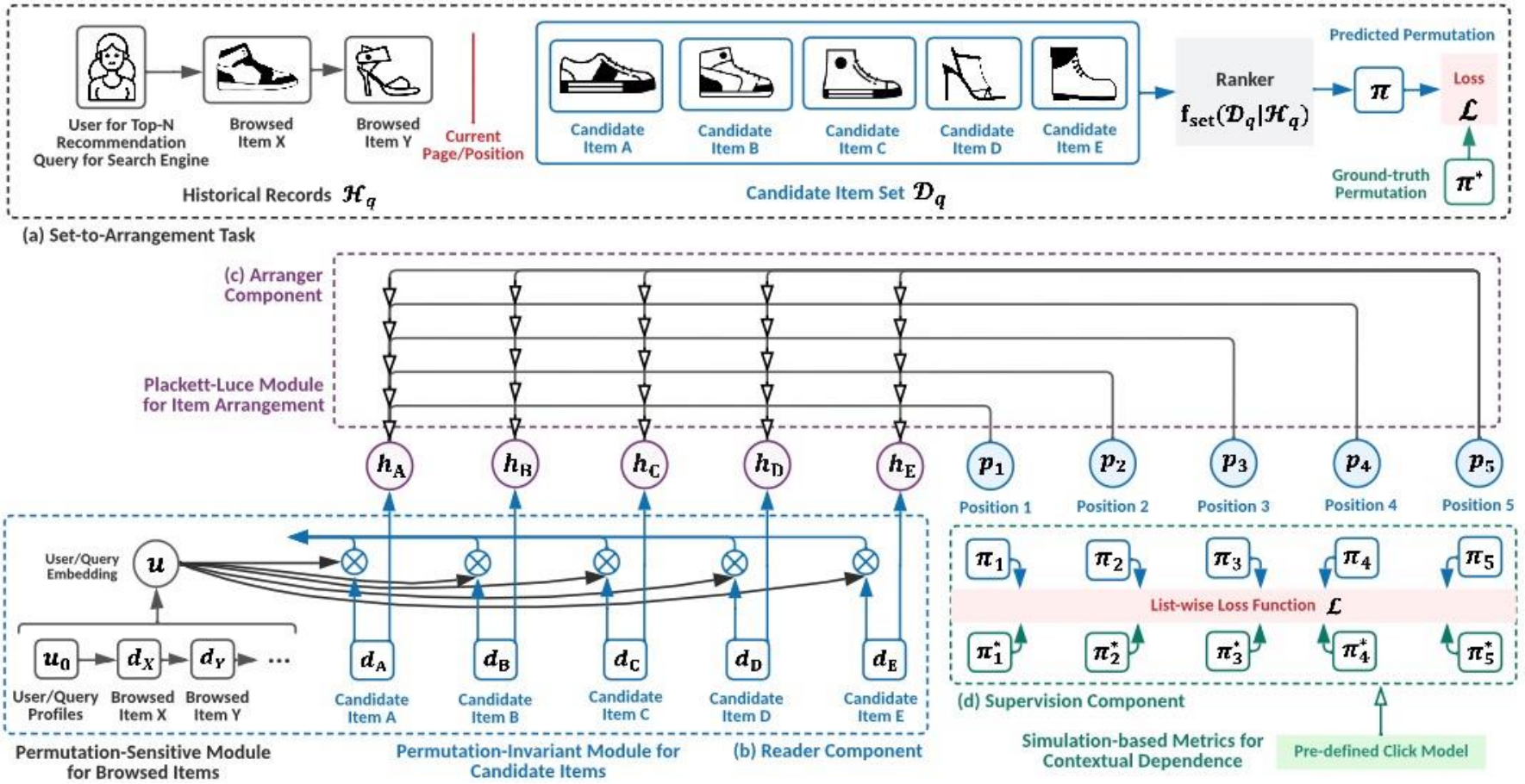

Figure: An illustrated example of STARank: (b) the reader module, (c) the arranger module, (d) the supervision module.

Reader

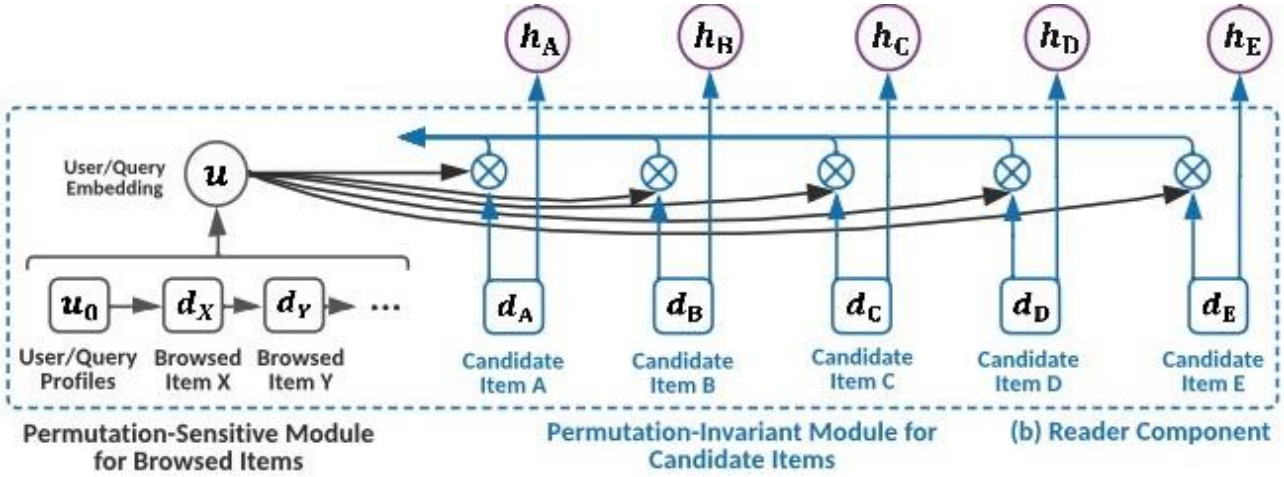

Figure: An illustrated example of the Reader module.

- We introduce a Permutation-Sensitive module to represent the user historical logs and the user embedding in a recursive manner as:

$$\boldsymbol{u}_q = \mathrm{PS}(\boldsymbol{u}_0, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_{|\mathcal{H}_q|})$$

where $\boldsymbol{u}_0$ is the embedding vector for user q: the user profiles (e.g., gender, age) as the original user feature. While practical, we adopt a RNN (i.e., LSTM) to form the PS module.

Reader

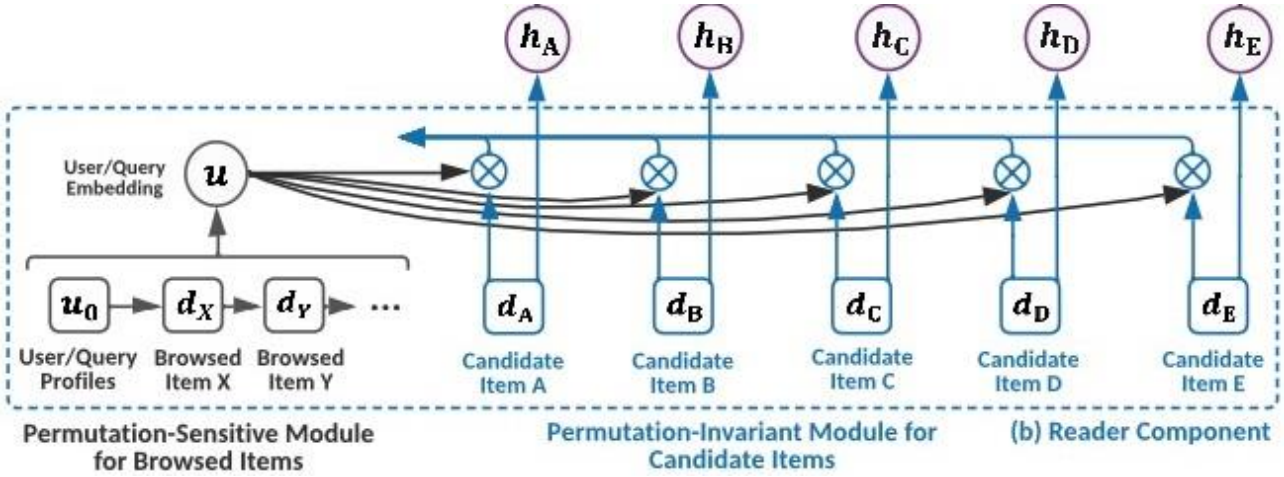

Figure: An illustrated example of the Reader module.

- We introduce a Permutation-Invariant module to encode the candidate items in $\mathcal{D}_q$:

$$\{\boldsymbol{h}_d | d \in \mathcal{D}_q\} = \mathrm{PI}(\{\boldsymbol{x}_d | d \in \mathcal{D}_q\}, \boldsymbol{u}_q)$$

where $\boldsymbol{h}_d$ is the representation vector of item d. In particular, our PI module is based on an attention mechanism.

Reader

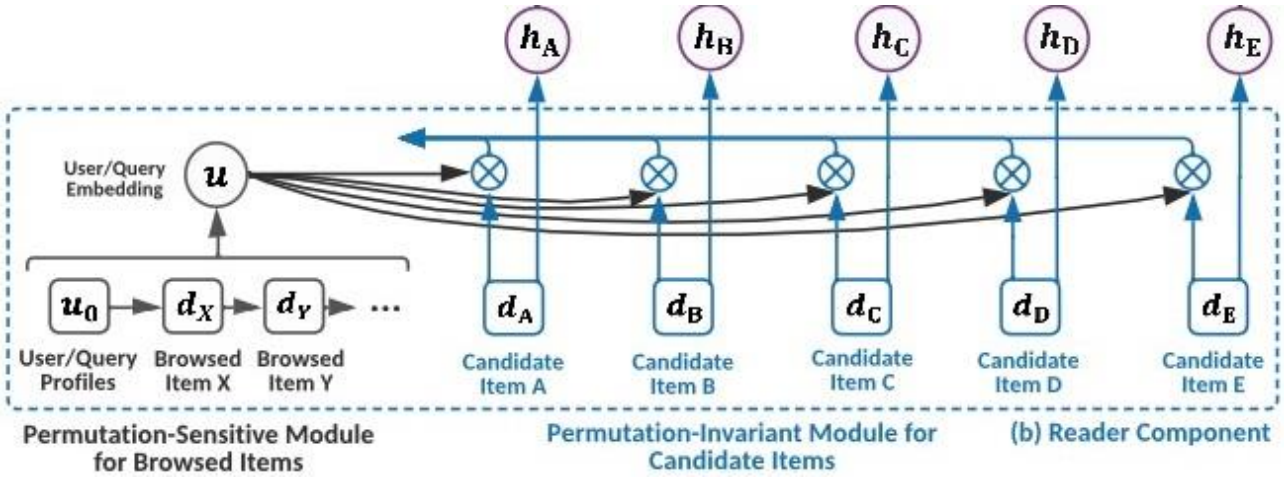

Figure: An illustrated example of the Reader module.

$$\{\boldsymbol{h}_d | d \in \mathcal{D}_q\} = \mathrm{PI}(\{\boldsymbol{x}_d | d \in \mathcal{D}_q\}, \boldsymbol{u}_q)$$

- The representation vector of each item d is produced by:

$$\boldsymbol{h}'_d = \boldsymbol{w}_1^{\mathrm{T}} \tanh(\boldsymbol{W}_1 \boldsymbol{x}_d + \boldsymbol{b}_1), \qquad d \in \mathcal{D}_q$$

$$\boldsymbol{h}_d = \beta_d \boldsymbol{h}'_d, \qquad \text{where } \beta_d = \frac{\boldsymbol{h}'_d \boldsymbol{u}_q^{\mathrm{T}}}{\sum_{j \in \mathcal{D}_q} \boldsymbol{h}'_j \boldsymbol{u}_q^{\mathrm{T}}}$$

where $\boldsymbol{w}$, $\boldsymbol{W}$, $\boldsymbol{b}$ are trainable matrices and vectors.

Arranger

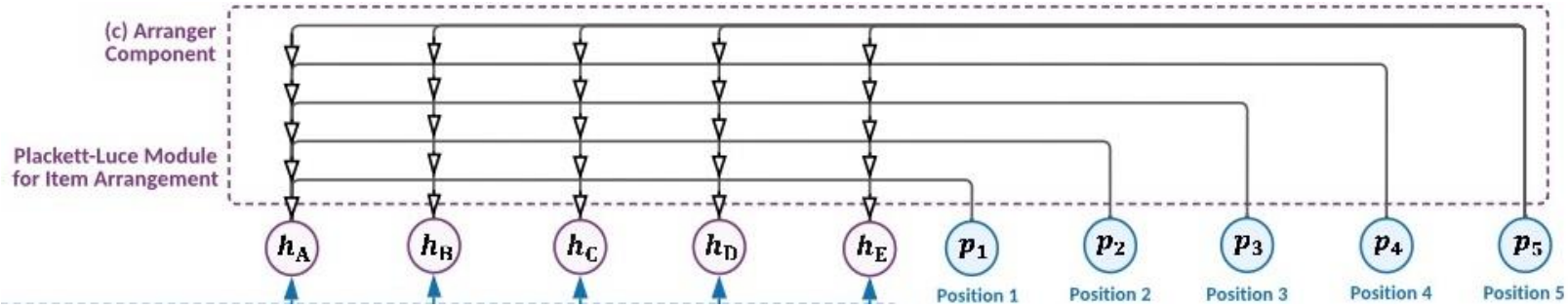

Figure: An illustrated example of the Arranger module.

- We use a Plackett-Luce (PL) module to generate their permutation. Formally, the PL module is defined as:

$$\pi = \mathrm{PL}(\{\boldsymbol{h}_d | d \in \mathcal{D}_q\}, \boldsymbol{u}_q)$$

Noticing that original pointer network using LSTM as the encoder is permutation-sensitive, we tweak the pointer network by replacing the LSTM with aforementioned reader.

Arranger

- We construct a batch of one-hot position embedding vector $\{\boldsymbol{p}_i\}_{i=1}^{|\mathcal{D}_q|}$ to encode the position information. These embeddings can be obtained by applying a LSTM as the encoder.
- Then, the probability distribution of item d at the i-th position can be produced as

$$s_d^i = \boldsymbol{v}_d^i \boldsymbol{u}_q^{\mathrm{T}}, \qquad \text{where } \boldsymbol{v}_d^i = \boldsymbol{w}_2^{\mathrm{T}} \tanh(\boldsymbol{W}_2 \boldsymbol{h}_d + \boldsymbol{W}_3 \boldsymbol{p}_i + \boldsymbol{b}_2)$$

$$P(\pi_i = d | \pi_{<i}) = \begin{cases} e^{s_d^i} / \sum_{k \in \mathcal{D}_q \setminus \pi_{<i}} e^{s_k^i} & \text{if } d \notin \pi_{<i} \\ 0 & \text{otherwise} \end{cases}$$

where $i = 1, \ldots, |\mathcal{D}_q|$. s_d^i is the trainable attention score associated with placing item d at position i and $p_d^i = P(\pi_i = d | \pi_{<i})$ is the probability representing the degree to which the PL module points item d to position i. p_d^i is computed by a softmax over the remaining items.

Supervisions

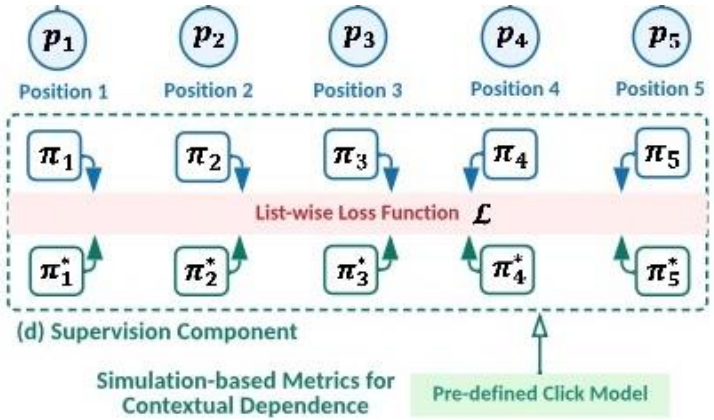

Figure: STARank is supervised by optimizing a list-wise loss function over π and the oracle permutation π^*. π^* is generated by a family of simulation-based ranking metrics.

- Generations should be supervised by:

$$\ell_{\text{list}}(\pi, \pi^*) = \sum_{i=1}^{|\mathcal{D}_q|} \ell_{\text{point}}(\pi_i^* | \pi_{<i}, \pi_i | \pi_{<i})$$

However, $\pi_i^* | \pi_{<i}$ is not available, since we only have the ground-truth permutation π^*. Alternatively, we use:

$$\ell_{\text{list}}(\pi, \pi^*) = \sum_{i=1}^{|\mathcal{D}_q|} \ell_{\text{point}}(\pi_i^* | \pi_{<i}^*, \pi_i | \pi_{<i}^*)$$

Supervisions

LEMMA 3.1. *Given a set of items $\mathcal{D}_q$ and one subset $\mathcal{D}'_q \subseteq \mathcal{D}_q$, the internal permutation of the items in $\mathcal{D}'_q$, denoted as π', is consistent in the context of either $\mathcal{D}_q$ or $\mathcal{D}'_q$, which can be formulated as*

$$P(\pi'|\mathcal{D}_q) = P(\pi'|\mathcal{D}'_q), \tag{14}$$

where $P(\pi'|\mathcal{D}_q)$ and $P(\pi'|\mathcal{D}'_q)$ are the probabilities of the items in $\mathcal{D}'_q$ satisfying π' in the context of using the candidate item sets $\mathcal{D}_q$ and $\mathcal{D}'_q$ as inputs respectively.

- The relative permutation of π_i and π^*_{i-1} is independent of how positions from 1 to $i-2$ are arranged.

$$P(\{\pi_i\} \cup \{\pi^*_{i-1}\}) = P(\{\pi_i\} \cup \{\pi^*_{i-1}\}|\pi^*_{<i-1})$$

Let $\pi^*_0 = \emptyset$, we have:

$$\ell_{\text{list}}(\pi, \pi^*) = \sum_{i=1}^{|\mathcal{D}_q|} \ell_{\text{point}}(\pi^*_i|\pi^*_{i-1}, \pi_i|\pi^*_{i-1})$$

Supervisions

- We propose to construct a virtual user to consider the contextual dependence to enumerate all the possible permutations $\pi \in \mathcal{P}$. We select the one received the highest score in terms of $\mathrm{R}(\cdot)$ as the oracle permutation π^*. We have:

$$\pi^* = \arg\max_{\pi \in \mathcal{P}} \mathrm{R}(\pi)$$

We use CM to denote an arbitrary click model:

$$\mathrm{R}_{\mathrm{CM}}(\pi) = \sum_{i=1}^{|\mathcal{D}_q|} P(c_{\pi_i} | \pi_{<i}) = \sum_{i=1}^{|\mathcal{D}_q|} P(o_{\pi_i} | \pi_{<i}) \cdot P(r_{\pi_i} | \pi_{<i})$$

NDCG is a special case of PBM:

$$\mathrm{R}_{\mathrm{NDCG}}(\pi) = \frac{1}{N} \sum_{i=1}^{|\mathcal{D}_q|} \left(\frac{1}{\log_2(i+1)}\right) \cdot (2^{r_{\pi_i}} - 1)$$

Set-to-Arrangement Framework

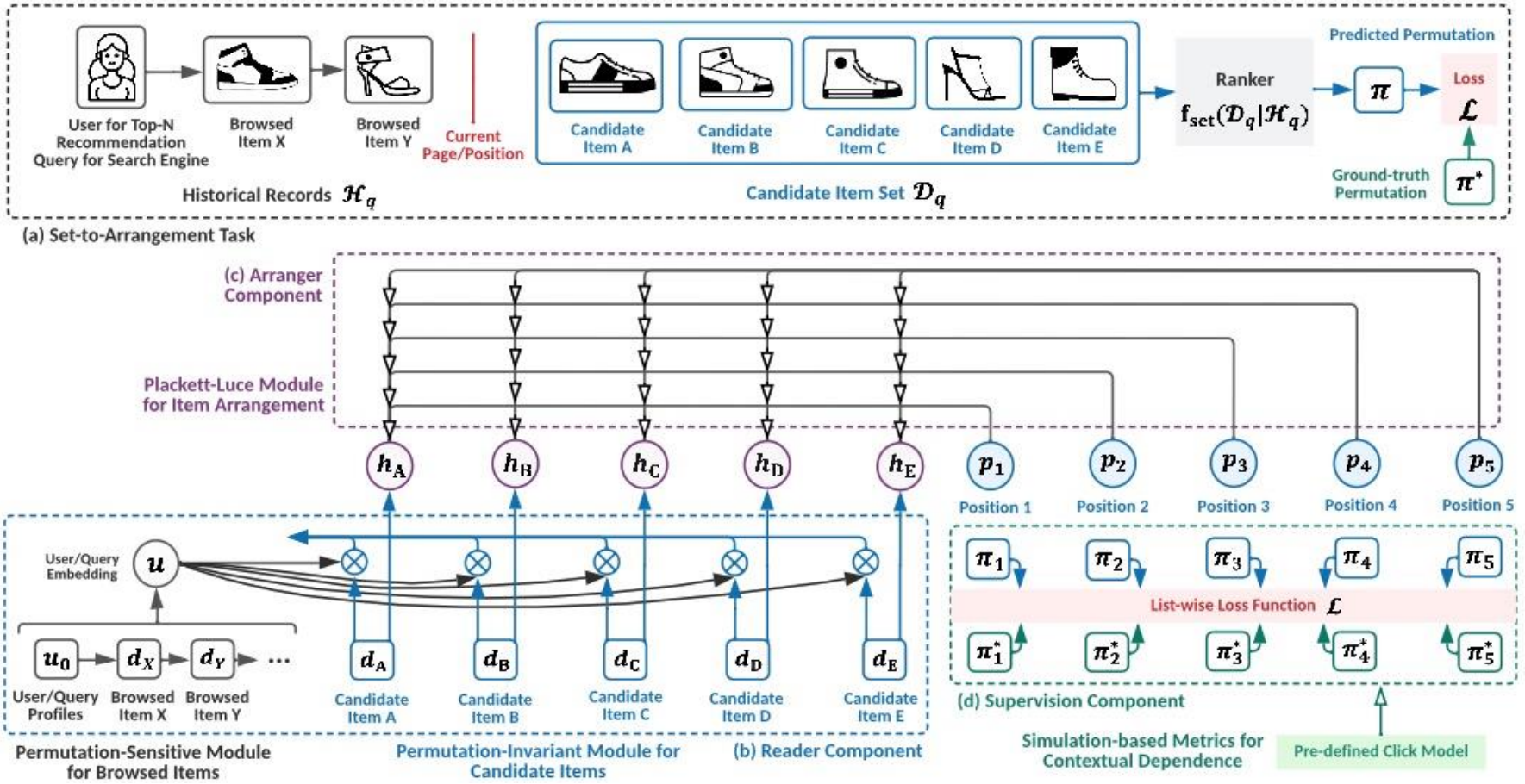

Figure: An illustrated example of STARank: (b) the reader module, (c) the arranger module, (d) the supervision module.

Content

- Problem Background
  - Set-to-Arrangement Framework
- Architecture
  - STARank
  - Supervisions
- **Experiment**
- Conclusion

Experiments

Table 2: Comparison of different rankers on three industrial top-N recommendation datasets in terms of NDCG and MAP at positions 5, 10. * indicates $p < 0.001$ in significance tests compared to the best baseline.

Ranker	Tmall				Alipay				Taobao			
	N@5	N@10	M@5	M@10	N@5	N@10	M@5	M@10	N@5	N@10	M@5	M@10
FM	0.1233	0.1140	0.2494	0.2572	0.1342	0.1277	0.2634	0.2709	0.1124	0.1106	0.2228	0.2354
DeepFM	0.1241	0.1181	0.2515	0.2591	0.1276	0.1214	0.2551	0.2629	0.1117	0.1104	0.2209	0.2339
PNN	0.1248	0.1185	0.2525	0.2602	0.1296	0.1227	0.2601	0.2666	0.1145	0.1123	0.2266	0.2385
LSTM	0.1341	0.1287	0.2660	0.2716	0.1427	0.1348	0.2783	0.2816	0.1308	0.1284	0.2496	0.2587
GRU	0.1311	0.1255	0.2606	0.2668	0.1422	0.1345	0.2773	0.2810	0.1300	0.1275	0.2488	0.2578
DIN	0.1345	0.1305	0.2674	0.2725	0.1401	0.1365	0.2705	0.2751	0.1230	0.1227	0.2351	0.2457
DIEN	0.1243	0.1182	0.2518	0.2594	0.1351	0.1293	0.2632	0.2696	0.1132	0.1122	0.2245	0.2366
SetRank	0.2733	0.2640	0.4090	0.4284	0.2969	0.2850	0.4289	0.4458	0.2594	0.2498	0.3876	0.4062
Seq2Slate	0.2385	0.2340	0.3720	0.3948	0.2452	0.2412	0.3771	0.3993	0.2223	0.2180	0.3437	0.3690
$\text{STARank}^{-}_{\text{PI}}$	0.3042	0.3251	0.4095	0.4254	0.3214	0.3264	0.4014	0.4446	0.2632	0.2734	0.3974	0.4214
$\text{STARank}^{-}_{\text{PS}}$	0.2952	0.3052	0.3974	0.4035	0.3046	0.3035	0.4046	0.4256	0.2678	0.2742	0.4025	0.4264
STARank	**0.3353***	**0.3745***	**0.4328***	**0.4358***	**0.4509***	**0.4803***	**0.5337***	**0.5703***	**0.3479***	**0.4278***	**0.4082***	**0.4635***
$\textbf{STARank}^{+}_{\text{PBM}}$	**0.3591***	**0.4311***	**0.4472***	**0.4932***	**0.4725***	**0.5046***	**0.5402***	**0.5854***	**0.4006***	**0.4868***	**0.4683***	**0.5250***
$\textbf{STARank}^{+}_{\text{UBM}}$	**0.3482***	**0.4208***	**0.4359***	**0.4828***	**0.4481***	**0.4709***	**0.5189***	**0.5644***	**0.3930***	**0.4792***	**0.4545***	**0.5164***

Ranker	Yahoo			
	N@5	N@10	M@5	M@10
FM	0.2122	0.2434	0.3574	0.3629
DeepFM	0.2483	0.2490	0.3587	0.3662
PNN	0.2563	0.2693	0.3598	0.3602
LSTM	0.2763	0.2731	0.3602	0.3641
GRU	0.2901	0.2876.	0.3674	0.3675
DIN	0.2932	0.2975.	0.3772	0.3706
DIEN	0.2911	0.2983	0.3703	0.3688
SetRank	0.3053	0.3185	0.3932	0.3868
Seq2Slate	0.3121	0.3234	0.4001	0.3987
$\text{STARank}^{-}_{\text{PI}}$	0.3188	0.3210	0.4079	0.4154
$\text{STARank}^{-}_{\text{PS}}$	0.3110	0.3356	0.4139	0.4201
STARank	**0.3399***	**0.3525***	**0.4301***	**0.4398***
$\textbf{STARank}^{+}_{\text{PBM}}$	**0.3474***	**0.3612***	**0.4342***	**0.4450***
$\textbf{STARank}^{+}_{\text{UBM}}$	**0.3355***	**0.3543***	**0.4324***	**0.4463***

Ranker	LETOR			
	N@5	N@10	M@5	M@10
FM	0.1752	0.1721	0.3282	0.3292
DeepFM	0.1746	0.1689	0.3243	0.3204
PNN	0.1775	0.1739	0.3296	0.3301
LSTM	0.1851	0.1795	0.3405	0.3384
GRU	0.1739	0.1741.	0.3170	0.3205
DIN	0.1550	0.1671.	0.2815	0.2988
DIEN	0.1818	0.1811	0.3409	0.3402
SetRank	0.2035	0.2246	0.4045	0.4002
Seq2Slate	0.2436	0.2546	0.4122	0.4031
$\text{STARank}^{-}_{\text{PI}}$	0.2443	0.2568	0.4172	0.4075
$\text{STARank}^{-}_{\text{PS}}$	0.2467	0.2606	0.4231	0.4123
STARank	**0.2824***	**0.2622***	**0.4876***	**0.4529***
$\textbf{STARank}^{+}_{\text{PBM}}$	**0.2843***	**0.2646***	**0.4906***	**0.4573***
$\textbf{STARank}^{+}_{\text{UBM}}$	**0.2863***	**0.2668***	**0.4887***	**0.4587***

Experiments

- We use PBM and UBM as the click models and compute the cumulative click probability at positions 5, 10, denoted as P@K, U@K.

Table 5: Comparison of different rankers on three industrial top-N recommendation datasets in terms of the proposed simulation-based ranking metrics based on PBM and UBM at positions 5, 10. * indicates $p < 0.001$ in significance tests compared to the best baseline.

Ranker	Tmall				Alipay				Taobao			
	P@5	P@10	U@5	U@10	P@5	P@10	U@5	U@10	P@5	P@10	U@5	U@10
FM	0.1026	0.1027	0.1015	0.1016	0.1138	0.1157	0.1013	0.1014	0.1073	0.1067	0.1013	0.1017
DeepFM	0.1023	0.1023	0.1005	0.1008	0.1037	0.1040	0.1008	0.1009	0.1093	0.1097	0.1008	0.1015
PNN	0.1248	0.1185	0.1014	0.1016	o.1023	0.1021	0.1002	0.1005	0.1142	0.1155	0.101	0.1016
LSTM	0.2074	0.2734	0.1301	0.1460	0.2137	0.2068	0.1274	0.1488	0.2498	0.2922	0.2025	0.2842
GRU	0.2188	0.3066	0.1299	0.1437	0.2292	0.2208	0.1308	0.1512	0.2541	0.3023	0.1928	0.2747
DIN	0.2725	0.2474	0.2493	0.1818	0.2257	0.2333	0.1443	0.1648	0.2748	0.2741	0.2099	0.2846
DIEN	0.2520	0.3693	0.1014	0.1012	0.2366	0.2194	0.1281	0.1456	0.2345	0.2986	0.1891	0.2654
SetRank	0.3778	0.3973	0.3378	0.3650	0.4171	0.4330	0.3346	0.3584	0.3094	0.3191	0.3704	0.3820
Seq2Slate	0.3182	0.3455	0.3210	0.3482	0.3337	0.3600	0.3254	0.3533	0.3285	0.3541	0.3189	0.3470
$\text{STARank}^{-}_{\text{PI}}$	0.3962	0.3967	0.4346	0.4426	0.5025	0.5046	0.4325	0.4602	0.3429	0.3478	0.4385	0.4255
$\text{STARank}^{-}_{\text{PS}}$	0.3873	0.3863	0.4253	0.4371	0.5010	0.5024	0.4242	0.4523	0.3321	0.3368	0.4257	0.4211
STARank	**0.4240***	**0.4010***	**0.4435***	**0.4493***	**0.5283***	**0.5058***	**0.4578***	**0.4888***	**0.3529***	**0.3599***	**0.4418***	**0.4308***

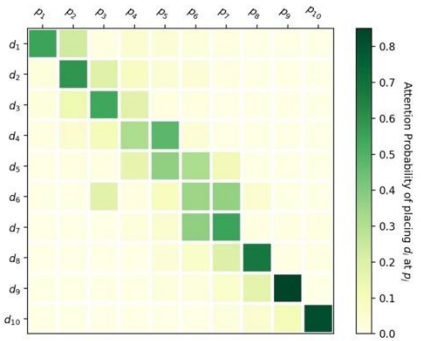

Figure 2: Visualization of attention probabilities of placing item d_i at position p_j on Alipay dataset.

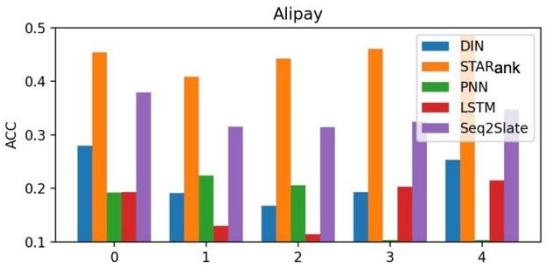

Figure 3: Performance comparisons of STARank against baselines in terms of accuracy at top 5 positions on Alipay dataset.

Content

- Problem Background
  - Set-to-Arrangement Framework
- Architecture
  - STARank
  - Supervisions
- **Experiment**
- Conclusion

Conclusion

- Our main contribution is to design a set-to-arrangement framework that allows the ranking models free from the sort operation: making the whole pipeline end-to-end differentiable.
- We design a new family of simulation-based supervisions directly over permutations instead of user feedbacks. These evaluation metrics (i.e., click models) can take the contextual dependence into consideration.
- No Scoring and Sorting, Let's Do Set-to-Arrangement!

Thanks for Your Listening

Slides Link

Jiarui's Wechat

Full Paper Link